UNIMINUTO
Corporación Universitaria Minuto de Dios
Educación de calidad al alcance de todos
Vigilada MinEducación

ALTA CALIDAD

Desarrollo de proyecto en Scratch

Sharith Vanessa Sanchez Rubio

Luisa Fernanda Quintero Martinez

1139057

1114686

Primer Semestre

Ingeniería de Sistemas, Facultad de Ingeniería, Corporación Universitaria Minuto de Dios

NRC 75299: Programación Básica

Instructor: Jasson Diaz Zamudio

Marzo 14, 2026

Introducción

El presente informe documenta el proceso de creación y replicación de un videojuego interactivo utilizando la plataforma Scratch. El objetivo principal de esta actividad fue aplicar conceptos de pensamiento computacional para dar vida a un personaje (un tripulante de *Among Us*), gestionando su movimiento, animaciones y respuestas al entorno. A través de este ejercicio, se exploraron componentes críticos del desarrollo de software, tales como el manejo de estados mediante variables, la toma de decisiones basada en estructuras condicionales y la modularización del código a través de bloques personalizados.

Contenido

DESARROLLO Y RESPUESTAS AL PROYECTO

Variables Utilizadas y su Función

En el proyecto se definieron variables clave para que el entorno no fuera estático:

- **Variable Velocidad:** Controla el desplazamiento del personaje.
    - *Función:* Define cuántos píxeles se mueve el personaje por cada pulsación de tecla.
    - *Bloque de modificación:* establecer [Velocidad v] a (5)
- **Variable Puntaje:** Gestiona el progreso del jugador.
    - *Función:* Acumular unidades cada vez que se interactúa con un objeto u objetivo.

- *Bloque de modificación:* cambiar [Puntaje v] en (1)

Afectación de las Variables en el Comportamiento

La variable Velocidad es el motor del movimiento. Al integrarla en los bloques de movimiento, cualquier cambio en su valor altera instantáneamente la agilidad del personaje en todo el código.

- **Fragmento de bloque:** mover (Velocidad) pasos

Estructura Condicional (Si / If)

Se utilizó una estructura si para gestionar los límites del escenario o colisiones.

- **Condición que evalúa:** ¿tocando el color [color de pared]?
- **Qué sucede:** * *Si se cumple:* El personaje rebota o se detiene (ejecuta un movimiento inverso).
    - *Si no se cumple:* El programa continúa con el siguiente bloque de código.
- **Bloque correspondiente:** ```scratch

si <¿tocando el color [#xxxxxx]?> entonces

mover ((-1) * (Velocidad)) pasos

fin

Control de Movimiento Direccional

Para que el personaje se mueva en las cuatro direcciones, se usaron bloques condicionales que detectan las entradas del teclado.

- **Descripción:** Cada dirección está ligada a un sensor de tecla presionada que cambia la orientación del personaje y su posición en los ejes X o Y.
- **Fragmento de bloque:**

Fragmento de código

```
si <¿tecla [flecha derecha v] presionada?> entonces
  apuntar en dirección (90)
  mover (Velocidad) pasos
fin
```

Ejemplo de Toma de Decisión (Colisión)

Un ejemplo crítico ocurre cuando el personaje intenta atravesar un obstáculo. El programa utiliza una condición de color o de objeto para decidir si permite el paso. Si la condición de "tocando pared" es verdadera, el juego anula el movimiento, obligando al usuario a buscar una ruta diferente.

Bloques Personalizados

Se crearon bloques para mejorar la legibilidad y evitar la repetición.

- **Función:** El bloque AnimarCaminata centraliza el cambio de disfraces.
- **Bloque creado:** definir AnimarCaminata -> siguiente disfraz, esperar (0.1) segundos.

Replicación del Código Original

La parte más importante para comprender el funcionamiento fue la **lógica de rotación y espejado**. Fue fundamental entender que para que el personaje no se pusiera de cabeza al ir a la izquierda, se debía fijar el estilo de rotación a "izquierda-derecha".

- **Fragmento equivalente:** fijar estilo de rotación a [izquierda-derecha v]

Innovación Agregada

Como innovación, añadí un **sistema de cambio de estado por velocidad**. Al recolectar un ítem, el personaje cambia de color usando efectos visuales y su variable de velocidad se duplica.

- **Bloques utilizados:** cambiar [efecto color v] en (25) y establecer [Velocidad v] a (10).

Dificultades y Soluciones

Durante el proceso surgieron tres dificultades técnicas principales:

1. **Error en la secuencia de disfraces:** El personaje mostraba un símbolo de exclamación en lugar de la animación de caminata. Esto ocurría porque el código estaba llamando al disfraz número 19 (exclamación) en lugar del disfraz inicial

(*standing*).

Solución: Se corrigió el índice del código para que apuntara al disfraz correcto de animación.

2. **Conflictos en la importación de fondos:** Al importar imágenes de la cafetería, el sistema les asignaba números adicionales (ej. "Cafetería 2") si se guardaban varias veces, lo que causaba que el código no encontrara el archivo correcto al intentar leer una ruta vacía o inexistente.

   Solución: Se renombraron los archivos y se aseguró que el código llamara exactamente al nombre del fondo guardado.

3. **Lógica de apariencia vs. fondo:** Se presentó un error donde se utilizaba el bloque cambiar fondo a en lugar de cambiar disfraz a, lo que impedía que el personaje cambiara de apariencia correctamente.

   Solución: Se reemplazaron los bloques de fondo por bloques de apariencia y se reubicó una variable que estaba mal posicionada en el flujo del programa.

Conclusiones

Modularidad: Aprendí que el uso de **bloques personalizados** es vital en ingeniería para que el código sea fácil de mantener y entender por otros desarrolladores.

Lógica Condicional: Se comprendió que los **condicionales** son los que otorgan "inteligencia" al software; sin ellos, no habría interacción real entre el usuario y el programa.

Gestión de Datos: El uso de **variables** permite crear programas flexibles. Al no usar valores fijos, podemos cambiar la dificultad o el comportamiento del juego simplemente modificando un dato inicial.